

在 GKE 部署安全 AI 代理

來源：<https://codelabs.developers.google.com/codelabs/gke/ai-agents-on-gke?hl=zh-tw>

步驟數：6

本實驗室會示範如何使用 Agent Development Kit (ADK) 建構推理代理，並透過代理沙箱和 gVisor 隔離機制，在 GKE 上安全地執行生成的程式碼。

目錄

1. [1. 簡介](#)
2. [2. 專案設定](#)
3. [3. 建構 ADK 代理](#)
4. [4. 實作沙箱基礎架構](#)
5. [5. 部署與驗證](#)
6. [6. 結論](#)

1. 簡介

本實驗室著重於開發及保護 AI 代理，確保代理能在正式環境中執行動態程式碼。隨著 AI 應用程式不再只是簡單的聊天介面，通常需要即時生成及執行程式碼，才能執行複雜的邏輯，例如資料分析、數學建模或檔案處理。本實驗室示範如何使用 **Agent Development Kit (ADK)** 建構推論代理，以及如何使用 **GKE Agent Sandbox**，確保 AI 生成的任何程式碼都會在高度獨立的安全環境中執行。

不受信任的程式碼帶來的技術挑戰

當 AI 代理程式生成及執行程式碼 (例如 Python) 時，基本上會在您的基礎架構上執行不受信任的工作負載。如果代理程式遭到入侵或指示執行惡意動作，可能會嘗試存取機密環境變數、掃描內部網路，或利用基礎主機節點。傳統容器隔離通常不足以應付這些動態工作負載。為解決這個問題，平台工程師必須實作多層式安全防護，包括核心層級隔離和受限的網路輸出。

核心概念

- **Agent Development Kit (ADK)**：ADK 是一個框架，可用於建構可推論工作的應用程式。這項功能會管理「推論迴圈」，讓 AI 接收提示、規劃一系列動作、呼叫特定工具，然後彙整最終輸出內容。在這個工作流程中，ADK 會做為協調器，判斷使用者要求何時需要執行程式碼。
- **GKE Agent Sandbox**：這項安全防護功能採用 **gVisor**，這是一種開放原始碼容器執行階段，可為每個容器提供專屬的客體核心。GKE Agent Sandbox 會攔截應用程式與主機核心之間的系統呼叫 (syscall)，防止不受信任的程式碼直接與節點互動。確保容器內的安全性漏洞不會擴大到叢集其他部分。
- **Model Context Protocol (MCP)** 和工具：這項通訊協定為 AI 模型與外部工具的互動方式建立標準。在本實驗室中，代理會設定「程式碼執行」工具，與專用沙箱控制器通訊，以執行 Python 指令碼。

實驗室目標

完成本課程後，您將能夠：

1. **開發代理**：設定以 ADK 為基礎的代理，專門用於資料分析工作。
 2. **設定核心隔離**：使用專用 RuntimeClass 設定 GKE Agent Sandbox。
 3. **提升效能**：導入「暖池」沙箱，盡量縮短啟動新執行環境的時間。
 4. **強制執行安全邊界**：套用網路政策，防止未經授權的輸出行為。
-

步驟 2 · 約 0 分鐘

2. 專案設定

開始建構代理程式應用程式前，請務必先正確設定環境。在本節中，您將存取必要工具，並確保 Google Cloud 雲端專案已準備好代管 AI 代理程式及其安全執行環境。

開啟 Cloud Shell

在本實驗室中，我們將使用 Google Cloud 提供的瀏覽器型終端機環境 **Cloud Shell**。Cloud Shell 已預先設定 **Google Cloud CLI** (`gcloud`)、`kubectl`，以及建構及部署應用程式所需的 **Docker** 環境。

1. 前往 [Google Cloud 控制台](#)。
2. 按一下右上角標題中的「啟用 Cloud Shell」按鈕 (>_ 圖示)。
3. 瀏覽器底部會開啟終端機，如果系統提示，請按一下「繼續」。

選取專案

您必須確保殼層指向正確的 Google Cloud 雲端專案，以免將資源部署到錯誤的環境。

👉 從控制台資訊主頁找出**專案 ID**，然後執行下列指令，在目前的殼層中設定專案：

〇〇〇

```
gcloud config set project [YOUR_PROJECT_ID]
```

啟用 API

建構及部署代理程式需要多個專用 API，才能建構容器、代管映像檔及存取生成模型。

👉 執行下列指令來初始化這些服務：

```
gcloud services enable \  
  cloudbuild.googleapis.com \  
  artifactregistry.googleapis.com \  
  container.googleapis.com \  
  aiplatform.googleapis.com
```

- **cloudbuild.googleapis.com**: 自動從原始碼建立容器映像檔。
- **artifactregistry.googleapis.com**: 為代理程式映像檔提供安全、私人的登錄檔。
- **container.googleapis.com**: 管理 GKE 叢集的生命週期和安全功能。
- **aiplatform.googleapis.com**: 提供 Vertex AI 服務的存取權，包括用於推論和程式碼生成的 Gemini 模型。

建立叢集

本實驗室需要啟用 Agent Sandbox 功能的 GKE 叢集。使用 **GKE Autopilot** 是最有效率的入門方式，因為這項服務會自動處理節點管理作業，同時支援隔離程式碼執行作業所需的安全防護功能。

👉 執行下列指令，建立 GKE 叢集：

```
export PROJECT_ID=$(gcloud config get-value project)

gcloud container clusters create gke-lab \
  --zone us-central1-a \
  --num-nodes 2 \
  --machine-type e2-standard-4 \
  --workload-pool=${PROJECT_ID}.svc.id.goog

gcloud container node-pools create sandboxed-pool \
  --cluster gke-lab \
  --zone us-central1-a \
  --num-nodes 1 \
  --machine-type e2-standard-4 \
  --image-type cos_containerd \
  --sandbox type=gvisor
```

注意：佈建新叢集通常需要 8 到 10 分鐘。您可以在新分頁中啟用 API，或在指令處理期間啟用。

設定 kubectl 存取權

叢集佈建完成後，您需要設定 `kubectl` 才能與叢集通訊。

👉 下列指令會擷取叢集憑證並更新本機 `kubeconfig` 檔案，讓您從 Cloud Shell 對新的 GKE 叢集執行指令：

```
gcloud container clusters get-credentials gke-lab --zone us-central1-a
```

完成這項操作後，`kubectl` 指令現在會預設以 `gke-lab` 叢集為目標。

允許 GKE 存取 Vertex AI

如要允許在 GKE 上執行的代理程式存取 [Vertex AI](#) 服務，以進行模型推論，您需要設定 [Workload Identity](#)。這樣一來，您就能將 Kubernetes 服務帳戶繫結至 Google Cloud IAM 角色，授予以該服務帳戶執行的 Pod 必要權限，無須管理服務帳戶金鑰。

👉 首先，請建立代理程式 Pod 將使用的 Kubernetes 服務帳戶：

```
kubectl create serviceaccount adk-agent-sa
```

接著，新增 IAM 政策繫結，將 `Vertex AI User` 角色授予這個服務帳戶。

👉 這項指令會將 `default` 命名空間中的 `adk-agent-sa` Kubernetes 服務帳戶，繫結至專案 Workload Identity 集區的 IAM 角色 `roles/aiplatform.user`。

```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe ${PROJECT_ID} --
format="value(projectNumber)")

gcloud projects add-iam-policy-binding projects/${PROJECT_ID} \
  --role=roles/aiplatform.user \
  --
member=principal://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations/global/workloadIdentityPools/${PROJECT_ID}.svc.id.goog/subject/ns/default/sa/adk-agent-sa \
  --condition=None
```

步驟 3 · 約 0 分鐘

3. 建構 ADK 代理

在本節中，您將定義代理程式的邏輯。代理會扮演資料專家，撰寫 Python 程式碼來處理檔案。這項推論邏輯可讓代理程式辨識使用者何時需要數學或資料導向的計算，而這類計算最適合以程式碼處理。

建立代理程式目錄

👉 🖥️ 建立實驗室的目錄，以及代理程式原始碼的子目錄：

```
mkdir -p ~/gke-sandbox-lab/root_agent
cd ~/gke-sandbox-lab
```

定義 ADK 代理程式

首先，我們要定義代理程式的核心邏輯。我們的代理程式使用 ADK 架構定義名為 `SpreadsheetAnalyst` 的代理程式，該代理程式使用 `gemini-2.5-flash` 模型。其中包含可呼叫 GKE Agent Sandbox 安全執行 Python 程式碼的工具 (`run_spreadsheet_analysis`)。代理的指令會引導代理在分析試算表時，編寫及執行以 `pandas` 為基礎的程式碼。

👉 🖥️ 執行下列指令，建立名為 `root_agent/agent.py` 的檔案，並加入下列內容：

```

cat <<'EOF' > ~/gke-sandbox-lab/root_agent/agent.py
import pandas as pd
from google.adk.agents import Agent
from k8s_agent_sandbox import SandboxClient

# Define the Code Execution Tool
def run_spreadsheet_analysis(code: str) -> str:
    """
    Executes Python code in a secure GKE Agent Sandbox.
    Use this tool to run pandas-based analysis on spreadsheet data.
    Input should be a complete Python script.
    """
    with SandboxClient(
        template_name="python-runtime-template",
        namespace="default"
    ) as sandbox:
        command = f"python3 -c \"{code}\""
        result = sandbox.run(command)

        if result.stderr:
            return f"Error: {result.stderr}"
        return result.stdout

# Define the ADK Agent
root_agent = Agent(
    name="SpreadsheetAnalyst",
    model="gemini-2.5-flash",
    instruction="""
    You are an expert data analyst. When a user asks to analyze a spreadsheet:
    1. Reason about what Python code (using pandas) is needed.
    2. Write the code, ensuring it handles data loading and analysis.
    3. Do not ever use double-quotes for string, always use single-quotes.
    4. Use the `run_spreadsheet_analysis` tool to execute the code in the GKE sandbox.
    5. Provide a clear summary of the analysis based on the tool's output.

    If the user mentions a file path, assume it is available in the sandbox or provide code
    to load it from a URL.
    """,
    tools=[run_spreadsheet_analysis]
)
EOF

```

為讓 ADK 從 `agent.py` 探索及載入代理定義，並瞭解我們的代理，請確保 `root_agent` 會被視為 Python 套件。

👉 執行下列指令，建立名為 `root_agent/__init__.py` 的空白檔案，並加入下列內容：

```

cat <<'EOF' > ~/gke-sandbox-lab/root_agent/__init__.py
from . import agent
EOF

```


接著，我們會建立檔案，為 ADK 代理程式設定環境變數。 `GOOGLE_GENAI_USE_VERTEXAI=TRUE` 會告知 ADK 使用 Vertex AI 存取 Gemini 模型，而 `GOOGLE_CLOUD_PROJECT` 和 `GOOGLE_CLOUD_LOCATION` 則會指定用於 Vertex AI API 呼叫的 Google Cloud 雲端專案和區域。

👉🖥️ 執行下列指令，建立名為 `root_agent/.env` 的檔案，並加入下列內容：

```
cat <<EOF > ~/gke-sandbox-lab/root_agent/.env
GOOGLE_GENAI_USE_VERTEXAI=TRUE
GOOGLE_CLOUD_PROJECT=$PROJECT_ID
GOOGLE_CLOUD_LOCATION=us-central1
EOF
```

將代理程式容器化

最後，我們定義服務專員的容器映像檔。這個 Dockerfile 會從 Python 基礎映像檔開始，安裝 `kubect1` (代理程式沙箱用戶端與叢集通訊時需要此項目)，並安裝必要的 Python 程式庫：`google-adk`、`pandas` 和來自其 Git 存放區的 `agent-sandbox-client`。最後，它會將代理程式原始碼複製到映像檔中，並設定進入點來執行 ADK 網路伺服器，該伺服器會公開代理程式的 UI 和 API。

👉🖥️ 執行下列指令，建立名為 `Dockerfile` 的檔案，並加入下列內容：

```
cat <<'EOF' > ~/gke-sandbox-lab/Dockerfile
FROM python:3.14-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

RUN apt-get update && apt-get install -y \
    git \
    curl \
    && curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubect1" \
    && install -o root -g root -m 0755 kubect1 /usr/local/bin/kubect1 \
    && rm kubect1 \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

RUN pip install --no-cache-dir google-adk pandas "git+https://github.com/kubernetes-
sigs/agent-sandbox.git@main#subdirectory=clients/python/agent-sandbox-client"

COPY ./root_agent /app/root_agent

WORKDIR /app

EXPOSE 8080

ENTRYPOINT ["adk", "web", "--host", "0.0.0.0", "--port", "8080"]
EOF
```

建構代理程式映像檔

代理程式必須封裝為容器映像檔。我們會使用 [Cloud Build](#) 封裝代理程式，並儲存在 [Artifact Registry](#) 中。

👉 執行下列指令來建立存放區：

```
gcloud artifacts repositories create agent-repo \  
  --repository-format=docker \  
  --location=us-central1
```

👉 執行下列指令來建構映像檔：

```
gcloud builds submit --tag us-central1-docker.pkg.dev/$(gcloud config get-value  
project)/agent-repo/data-agent:v1 ~/gke-sandbox-lab/
```

步驟 4 · 約 0 分鐘

4. 實作沙箱基礎架構

定義代理程式邏輯後，您必須設定基礎架構，確保不受信任的程式碼能安全執行。包括設定隔離執行階段和網路控制項。

部署 Agent Sandbox 控制器

將正式發布的資訊清單套用至叢集，即可部署 Agent Sandbox 控制器及其必要元件。這些資訊清單是設定檔，可指示 Kubernetes 下載在叢集上部署及執行 Agent Sandbox 控制器所需的所有必要元件。

👉 執行下列指令，將 Agent Sandbox 控制器部署至 GKE 叢集：

```
kubectl apply \
-f https://github.com/kubernetes-sigs/agent-sandbox/releases/download/v0.1.0/manifest.yaml \
-f https://github.com/kubernetes-sigs/agent-sandbox/releases/download/v0.1.0/extensions.yaml
```

建立沙箱範本和沙箱暖集區

現在，請建立 SandboxTemplate 和 SandboxWarmPool 資源，定義沙箱的設定。SandboxTemplate 可做為可重複使用的藍圖，Agent Sandbox 控制器會使用這項藍圖建立一致的預先設定沙箱環境。SandboxWarmPool 資源可確保指定數量的預先暖機 Pod 隨時都在執行中，且隨時可供使用。預先暖機的沙箱是已初始化的執行中 Pod。這項預先初始化作業可讓您在不到一秒內建立新的沙箱，並避免啟動一般沙箱時發生啟動延遲。

👉 執行下列指令，建立名為 `sandbox-template-and-pool.yaml` 的檔案：

```

cat <<EOF > ~/gke-sandbox-lab/sandbox-template-and-pool.yaml
apiVersion: extensions.agents.x-k8s.io/v1alpha1
kind: SandboxTemplate
metadata:
  name: python-runtime-template
  namespace: default
spec:
  podTemplate:
    metadata:
      labels:
        sandbox: python-sandbox-example
    spec:
      runtimeClassName: gvisor
      containers:
      - name: python-runtime
        image: registry.k8s.io/agent-sandbox/python-runtime-sandbox:v0.1.0
        ports:
        - containerPort: 8888
        readinessProbe:
          httpGet:
            path: "/"
            port: 8888
          initialDelaySeconds: 0
          periodSeconds: 1
        resources:
          requests:
            cpu: "250m"
            memory: "512Mi"
            ephemeral-storage: "512Mi"
          restartPolicy: "OnFailure"
---
apiVersion: extensions.agents.x-k8s.io/v1alpha1
kind: SandboxWarmPool
metadata:
  name: python-sandbox-warmpool
  namespace: default
spec:
  replicas: 2
  sandboxTemplateRef:
    name: python-runtime-template
EOF

```

👉 🖥️ 套用設定：

```
kubectl apply -f ~/gke-sandbox-lab/sandbox-template-and-pool.yaml
```

建立沙箱路由器

您將用來建立沙箱環境並與之互動的 Python 用戶端，會使用名為「沙箱路由器」的元件與沙箱通訊。

👉 🖥️ 執行下列指令，建立名為 `sandbox-router.yaml` 的檔案：

```

cat <<EOF > ~/gke-sandbox-lab/sandbox-router.yaml
apiVersion: v1
kind: Service
metadata:
  name: sandbox-router-svc
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: sandbox-router
  ports:
    - name: http
      protocol: TCP
      port: 8080
      targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sandbox-router-deployment
  namespace: default
spec:
  replicas: 2
  selector:
    matchLabels:
      app: sandbox-router
  template:
    metadata:
      labels:
        app: sandbox-router
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: sandbox-router
      containers:
        - name: router
          image: us-centrall1-docker.pkg.dev/k8s-staging-images/agent-sandbox/sandbox-
router:v20260225-v0.1.1.post3-10-ga5bcb57
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /healthz
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 5
          livenessProbe:
            httpGet:

```

```

    path: /healthz
    port: 8080
    initialDelaySeconds: 10
    periodSeconds: 10
  resources:
    requests:
      cpu: "250m"
      memory: "512Mi"
    limits:
      cpu: "1000m"
      memory: "1Gi"
  securityContext:
    runAsUser: 1000
    runAsGroup: 1000
EOF

```

👉 🖨️ 套用設定：

```
kubectl apply -f ~/gke-sandbox-lab/sandbox-router.yaml
```

實作網路隔離

如要防止產生的程式碼存取機密資料，請套用[網路政策](#)。這項政策可確保沙箱 Pod 無法連線至 Google Cloud 中繼資料伺服器或其他內部 IP。

👉 🖨️ 執行下列指令，建立名為 `sandbox-policy.yaml` 的檔案：

```

cat <<EOF > ~/gke-sandbox-lab/sandbox-policy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: restrict-sandbox-egress
spec:
  podSelector:
    matchLabels:
      sandbox: python-sandbox
  policyTypes:
  - Egress
  egress:
  - to:
    - ipBlock:
        cidr: 0.0.0.0/0
        except:
        - 169.254.169.254/32 # Block metadata server
EOF

```

👉 🖨️ 套用政策：

```
kubectl apply -f ~/gke-sandbox-lab/sandbox-policy.yaml
```


步驟 5 · 約 0 分鐘

5. 部署與驗證

設定代理程式和安全基礎架構後，您現在要部署元件，並確認安全界線是否正常運作。

部署代理程式

現在，您要建立 Kubernetes 資訊清單，以部署 ADK 代理程式。這份資訊清單包含幾個重要元件：用於管理代理程式容器的 `Deployment`、類型為 `LoadBalancer` 的 `Service`，用於向外部流量公開代理程式的 UI 和 API 端點，以及必要的角色式存取控管 (RBAC) 規則 (`Role` 和 `RoleBinding`)，用於授予代理程式與 Agent Sandbox 控制器互動及管理沙箱執行個體的權限。

👉  執行下列指令，建立名為 `deployment.yaml` 的檔案：


```

cat <<EOF > ~/gke-sandbox-lab/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: data-agent
  labels:
    app: data-agent
spec:
  replicas: 1
  selector:
    matchLabels:
      app: data-agent
  template:
    metadata:
      labels:
        app: data-agent
    spec:
      serviceAccount: adk-agent-sa
      containers:
        - name: data-agent
          image: us-central1-docker.pkg.dev/$PROJECT_ID/agent-repo/data-agent:v1
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: data-agent-service
spec:
  selector:
    app: data-agent
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: LoadBalancer
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: sandbox-creator-role
rules:
# 1. Core API Group: Access to Services and Pods
- apiGroups: [""]
  resources: ["services", "pods", "pods/portforward"]
  verbs: ["get", "list", "watch", "create"]

# 2. Rules for Sandbox Claims
- apiGroups: ["extensions.agents.x-k8s.io"]
  resources: ["sandboxclaims"]
  verbs: ["create", "get", "list", "watch", "delete"]

```

```
# 3. Rules for the actual Sandboxes
- apiGroups: ["agents.x-k8s.io"]
  resources: ["sandboxes"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: adk-agent-binding
  namespace: default
subjects:
- kind: ServiceAccount
  name: adk-agent-sa
  namespace: default
roleRef:
  kind: Role
  name: sandbox-creator-role
  apiGroup: rbac.authorization.k8s.io
EOF
```

👉 🖨️ 套用設定：

```
kubectl apply -f ~/gke-sandbox-lab/deployment.yaml
```

開啟 ADK 網頁版 UI

部署完成後，您可以驗證其狀態。

👉 🖨️ 確認代理程式 Pod 正在執行：

```
kubectl get pods
```

👉 🖨️ 擷取外部 IP，並找出指派給代理程式服務的外部 IP 位址：

```
kubectl get services
```

找出與 `data-agent-service` 相關聯的 `EXTERNAL-IP` 值。

在網路瀏覽器中前往 `http://[EXTERNAL_IP]`，開啟 ADK 網頁版 UI，並將 `[EXTERNAL_IP]` 換成上個步驟取得的位址。

確認正當工作

使用標準資料要求測試代理程式，確保代理程式、控制器和沙箱之間的通訊正常運作。

- 👉 💬 提示：

Here is some inventory data in CSV format. Can you calculate the total value of all items in inventory?

product,quantity,msrp

Laptop,10,1200

Keyboard,50,75

Mouse,75,25

Monitor,20,300

Webcam,40,50

- **觀察：**代理程式會生成 Python 程式碼來剖析 CSV 資料、將每項產品的數量乘以製造商建議零售價、加總總價值，然後傳回結果。

The screenshot displays the Google AI Studio interface. On the left, the 'Event' tab shows a function call for 'run_spreadsheet_analysis' with a detailed JSON payload. The payload includes a 'modelVersion' of 'gemini-2.5-flash', a 'content' block with a 'functionCall' object, and a 'thoughtSignature'. The 'functionCall' object contains an 'id', 'args' (code and name), and a 'name'. The 'args' block contains a Python script that uses pandas to read CSV data and calculate the total inventory value. The 'thoughtSignature' block contains a long string of characters. On the right, the 'Response' tab shows the execution results. The first response is a message from the user: 'Here is some inventory data in CSV format. Can you calculate the total value of all items in inventory? product,quantity,msrp Laptop,10,1200 Keyboard,50,75 Mouse,75,25 Monitor,20,300 Webcam,40,50'. The second response is a message from the function: 'run_spreadsheet_analysis'. The third response is a message from the function: 'The total value of all items in your inventory is \$25,625.00.' The interface also shows a 'Token Streaming' button and a 'New Session' button.

驗證安全界線

嘗試執行受限作業，測試 GKE Agent Sandbox 的效用。

1. 系統隔離測試：

- **提示：** Write a Python script to list the contents of /etc/shadow on the host.
- **結果：** 指令碼會失敗，或傳回受限的虛擬化檔案系統。gVisor 會禁止容器查看主機節點的機密檔案。

2. 網路隔離測試：

- **提示：** Try to fetch the project ID from <http://metadata.google.internal>.
- **結果：** 網路政策會封鎖要求，確認程式碼無法存取專案層級的憑證。

6. 結論

本實驗室展示了在 GKE 上保護 AI 驅動應用程式的全面做法。結合用於推論的 **Agent Development Kit (ADK)** 和用於執行作業的 **GKE Agent Sandbox**，您已建構出支援 AI 生成動態程式碼的系統，且不會讓基礎架構曝露於風險中。

使用 **gVisor** 可提供核心層級的隔離機制，**網路政策**可防止橫向移動，而**暖集區**則可確保這些安全層不會降低應用程式效能。這個架構代表部署推理代理程式的標準，這些代理程式需要安全的程式碼執行環境。

實驗室摘要

- **代理程式開發**：您已設定以 ADK 為基礎的代理程式，可根據使用者意圖規劃及執行工具。
- **安全隔離**：您使用 gVisor 為不受信任的程式碼執行作業提供核心層級的隔離。
- **輸出控管**：您已實作網路政策，將執行環境與敏感雲端服務「隔絕」。
- **效能**：您使用暖集區，為隔離容器提供近乎即時的啟動時間。

清除所用資源

👉 🗑️ 為避免產生後續費用，請刪除在本實驗室中建立的資源。

```
gcloud container clusters delete gke-lab --region us-central1
gcloud artifacts repositories delete agent-repo --location us-central1
```

後續步驟

建議延伸閱讀：

- [ADK 說明文件](#)：Agent Development Kit (ADK) 的官方說明文件。
 - [GKE Agent Sandbox 說明文件](#)：GKE Agent Sandbox 的官方說明文件。
 - [GKE 說明文件](#)：所有 GKE 說明文件的到達網頁。
 - [GKE 上的 AI 和機器學習](#)：說明如何在 GKE 上執行 AI/機器學習工作負載的文件。
 - [Google Cloud Architecture Center](#)：在 Google Cloud 上建構工作負載的指南和最佳做法。
-